REFORGE: Automated Self-Healing Architecture for High-Throughput IoT Telemetry Pipelines

Section IV: System Implementation & Microservices Architecture Specification

SECTION IV: SYSTEM IMPLEMENTATION

This section presents the concrete architecture, algorithms, and execution workflows of the **REFORGE** system as implemented in the codebase. REFORGE is an automated, self-healing data pipeline framework designed for continuous Internet of Things (IoT) telemetry. It bridges deterministic domain rule verification, multivariate machine learning (ML) anomaly detection, sandbox-isolated data repair, and pipeline code governance.

A. System Architecture and Technology Stack

The REFORGE implementation adopts a containerized, decoupled microservices architecture orchestrated via Docker Compose (docker-compose.yml), comprising four network-isolated services operating over an internal bridge network (reforge-net):

- **Ingestion & Orchestration Engine (server)**: Implemented in Node.js (v20 LTS) with Express.js. It manages REST endpoints, orchestrates the two-stage validation pipeline, computes online temporal/cross-sensor features, drives the hybrid decision layer, and executes the data repair cascade.
- **Machine Learning Microservice (ml-service)**: Implemented in Python 3.11 with FastAPI and Uvicorn. It hosts scikit-learn, XGBoost, and LightGBM model runtimes, manages the serialized model registry (registry.json), exposes synchronous inference endpoints (POST /predict), and executes training/evaluation routines.
- **Primary Document Store (mongo)**: MongoDB 7.0 container maintaining immutable raw observation logs, validation results, quarantine queues, dataset version overlays, and append-only audit trails.
- **Operations & Governance Console (frontend)**: Single-Page Application (SPA) built with React 18, Vite, and Redux Toolkit, served through an Alpine Nginx reverse proxy. It provides operators with real-time pipeline monitoring, quarantine review, model comparison, and repair auditing.

```
IoT Telemetry Stream → HTTP POST /api/ingest → Node.js Orchestrator → MongoDB RawObservation (Immutable)
↓
Structural Schema Validation (Joi) → Feature Builder (Dual-Scale & Cross-Sensor)
↓
FastAPI ML Service (:8000) ←→ Calibrated Hybrid Decision Engine (v1.2.0) ←→ Deterministic Rule Engine
↓
Valid [0.0, 0.3) → Cleansed Stream | Quarantine Zone (Low-Risk ≤ 0.60 | High-Risk > 0.60)
↓
Iterative Multi-Strategy Sandbox Cascade Healing → Versioned Overlay
```

B. Ingestion Immutability and Structural Validation

Data enters through POST /api/ingest handled by validationPipeline.js. The platform enforces strict data immutability and idempotency through the following sequence:

1) **Idempotent Raw Ingestion**: Each payload is assigned a deterministic ingestion key: ingestKey = sourceId || entityId || timestamp. The raw payload is committed to MongoDB (RawObservation collection) prior to any analytical or transformation step. If a duplicate ingestKey is detected, the ingestion pipeline terminates early and returns the cached historical ValidationResult, preventing redundant processing.

2) **Envelope & Schema Validation**: Executed via Joi schemas in schemaValidator.js. It enforces required fields (sourceId, entityId, timestamp, values.value), ISO 8601 timestamp conformance, and valid numeric types. If structural validation fails, semantic rules and ML inference are bypassed, the record is immediately classified as invalid, and an immutable event is committed to AuditLog.

C. Feature Engineering and Semantic Context Construction

To detect context-dependent physical anomalies without database bottlenecks, featureBuilder.js derives multi-scale temporal and spatial features. It maintains two bounded, least-recently-used (LRU) in-memory caches: sensorHistoryCache (bounded to 500 sensors, storing the trailing 30 timestamps and values per entity) and peerLatestCache (retaining the most recent readings across spatial peers within an operating zone). For each incoming observation (t, v) of entity e = <zone>-<type>, the system computes:

- ***Cyclical Diurnal Temporal Encodings***: $\theta = 2\pi \times (hour_{UTC} + min/60 + sec/3600) / 24$; $timeOfDaySin = \sin(\theta)$, $timeOfDayCos = \cos(\theta)$.

- ***Dual-Scale Rolling Statistics***: *Fast window (N_{fast} = 5) and slow window (N_{slow} = 20) means and standard deviations (μ_{fast}, σ_{fast}, μ_{slow}, σ_{slow}).*
  volatilityRatio = max(σ_{fast}, 0.1) / max(σ_{slow}, 0.1), meanDivergence = μ_{fast} - μ_{slow}.
- ***Temporal Dynamics***: *$\Delta v = v_t - v_{t-1}$, $\Delta t_{min} = (t - t_{prev}) / 60000$;*
  historicalZScore = clip((v - μ_{slow}) / max(σ_{slow}, 0.1), -10.0, 10.0).
- ***Spatial Peer Consensus Context***: *Aggregates peers of identical sensor type within a temporal window W = ±3 hours. Given peer readings $\{p_1, ..., p_k\}$ ($k \geq 2$):*
  σ_{eff} = max(σ_{peer}, σ_{min}(type)), where σ_{min} enforces physical regularizers (0.6 for temp, 2.0 for humidity, 0.4 for pressure).
  normalizedPeerResidual = clip((v - μ_{peer}) / σ_{eff}, -10.0, 10.0).

D. Anomaly Detection and Hybrid Decision Engine

REFORGE executes a dual-engine architecture combining deterministic domain rules with supervised/unsupervised machine learning models.

1) **Deterministic Rule Engine & Risk Scoring**: ruleEngine.js evaluates four explainable rules producing severities $s_j \in [0, 1]$: (i) *Rate-of-Change Rule* triggers if $|\Delta v| / \sigma_{slow} > 4.0$; (ii) *Historical Z-Score Rule* triggers if |historicalZScore| > 3.0; (iii) *Cross-Sensor Disagreement Rule* triggers if |normalizedPeerResidual| > 3.0 with $k \geq 2$; and (iv) *Physically Implausible Jump Rule* triggers if $|\Delta v| > 8 \times \sigma_{slow}$ (or fallback absolute jump of 40.0 units). scoringEngine.js calculates composite risk $R_{rule} = \sum w_j \cdot s_j$ with weights **w** = $[0.20, 0.30, 0.25, 0.25]^T$. Boundaries map R_{rule} into: valid ($R < 0.30$), suspicious ($0.30 \leq R < 0.60$), and invalid ($R \geq 0.60$).

2) **Machine Learning Microservice**: Hosts models registered in registry.py: LightGBM and XGBoost classifiers trained on 33-dimensional telemetry vectors, unsupervised Isolation Forest measuring recursive path length, and Temporal Statistical anomaly detectors.

3) **Calibrated Hybrid Decision Engine (Policy v1.2.0)**: Implemented in hybridDecisionEngine.js, this layer reconciles rule-based and ML outputs under three operating modes: rule_only, shadow, and hybrid. In hybrid mode, consensus is governed by Table I.

TABLE I: Calibrated Hybrid Consensus Decision Matrix

Comparison Category	Rule Risk (R_{rule})	ML Score (S_{ML})	Official Class	System Action & Escalation Policy
rule_normal_ml_normal	< 0.30	< 0.50	**Valid**	Ingest directly into cleansed production telemetry stream.
rule_anomaly_ml_anomaly	≥ 0.30	≥ 0.50	**Invalid**	High-confidence consensus anomaly; isolate in Quarantine.
rule_anomaly_ml_normal	≥ 0.30	< 0.50	**Suspicious**	Physical guard breached; route to Quarantine for review.
rule_normal_ml_anomaly	< 0.30	≥ 0.80 or ≥ 0.60 w/ R ≥ 0.20	**Suspicious**	High-confidence ML multivariate shift; route to Quarantine.
rule_normal_ml_anomaly	< 0.30	< 0.80 & R < 0.20	**Valid**	Suppress noisy ML outlier; preserve rule-based baseline (0 FP).
ML Degraded / Timeout	Any	Timeout / 5xx	**Authoritative Rule**	Graceful fallback to rule classification with audit log alert.

E. Two-Tier Quarantine and Automated Data Healing Workflow (Mode A)

Observations classified as suspicious or invalid are intercepted by quarantine.controller.js and isolated from downstream consumers. The Quarantine Zone is partitioned into two operational tiers: (i) **Low-Risk Tier** ($R_{rule} \leq 0.60$ / Suspicious), representing moderate statistical anomalies eligible for single-click bulk automated healing (POST /api/quarantine/heal with tier: 'low_risk'); and (ii) **High-Risk Tier** ($R_{rule} > 0.60$ / Invalid), representing severe physical bounds violations or implausible step jumps requiring individual inspection or batch sandbox healing.

Multi-Strategy Healing Cascade: dataRepairService.js executes an iterative sandbox loop over prioritized heuristic estimators until a candidate yields $R_{repaired} < 0.30$ and passes ML consensus: (1) *Linear Temporal Interpolation* between predecessor and successor: $v_{interp} = v_{pred} + (v_{succ} - v_{pred}) \cdot \alpha$; (2) *Spatial Peer Cluster Median* across active peers in $[min_{phys}, max_{phys}]$; (3) *Device Rolling Median* from the preceding 10 normal observations; and (4) *Physical Domain Clamping* bounded strictly within sensor hardware limits $[min_{phys}, max_{phys}]$.

Versioned Dataset Overlays: In accordance with zero-data-loss architecture, raw observations are never overwritten. datasetVersionService.js creates an immutable DatasetVersion overlay storing the tuple ⟨rawId, v_{raw}, v_{healed}, strategy, timestamp,

actor⟩. Cleansed datasets are exported via streaming binary .csv, .xlsx, and vector .pdf. Furthermore, POST /api/export/github automatically commits reforge_healed_<version>.csv, reforge_repair_audit_report_<version>.md, and reforge_healed_<version>.json directly to the target GitHub repository (Kushal08-04/Reforge) under datasets/ via REST API.

F. Pipeline Code Defect Repair and Subprocess Sandbox Isolation (Mode B)

When an anomaly stems from a defect in the ETL codebase rather than a data anomaly (e.g., missing sensor unit conversion or unhandled edge cases), REFORGE invokes **Mode B: Code-Level Repair**:

1) **LLM Diagnostic Synthesis**: repairDiagnosisService.js and llmProvider.js synthesize structured JSON proposals specifying failure categories, diagnostic hypotheses, target files, unified code diffs, and declarative unit test assertions.

2) **Subprocess Sandbox Runner**: Handled by server/src/sandbox/runner.js, the candidate patch is written to an isolated directory and loaded using Node.js's CommonJS loader (strictly prohibiting arbitrary eval or new Function() invocations).

3) **Verification Gates**: patch_config.json enforces non-empty test suites, strict per-test execution timeouts (2500 ms), memory output limits (64 KB), non-NaN numeric validity, and raw value preservation. An exit code of 0 and 100% test passage are required to transition from PROPOSED to VERIFIED.

4) **Pull Request Governance**: githubService.js branches from main, commits the verified file, and opens a GitHub Pull Request with execution logs and risk disclosures. Merging is strictly gatekept by human review (PATCH /api/repairs/:id/review).

G. Operations Dashboard, Model Governance, and Audit Trail

The React dashboard interface (frontend/src/pages/) provides operational oversight across five core views: (1) *Pipeline Overview* (DashboardPage.jsx) renders live ingestion counters, classification breakdown (Valid, Suspicious, Invalid), Rule vs. ML agreement rates, and a classification bar chart; (2) *Rule vs. ML Comparison* (ComparisonPage.jsx) exposes a 2×2 contingency matrix tracking consensus and divergence; (3) *Model Management* (ModelManagementPage.jsx) provides one-click zero-downtime model promotion (POST /api/models/:version/promote) in registry.json; (4) *Evaluation Benchmark* (EvaluationPage.jsx) displays precision, recall, F1, specificity, FPR, ROC-AUC, and PR-AUC; and (5) *Append-Only Audit Lineage* (AuditPage.jsx) queries the AuditLog collection to verify every lifecycle event from ingestion to human approval.

H. Current Implementation Limitations and Incomplete Features

To maintain scientific integrity, the following architectural constraints of the current codebase are explicitly identified:

- **Protocol Ingestion Scope**: Ingestion is currently restricted to HTTP REST (POST /api/ingest). Distributed event streaming message brokers (e.g., Apache Kafka, MQTT, or AMQP) described in high-scale architectural roadmaps are not yet integrated.
- **Authentication and Access Governance**: While API payloads and database records accept actor metadata (reviewer, appliedBy), end-to-end authentication (JWT/OAuth2) and cryptographic Role-Based Access Control (RBAC) are not enforced at the network gateway; operations currently run with local development/bypass privileges.
- **In-Memory Caching Topology**: Sensor history buffers and peer caches in featureBuilder.js reside in Node.js process memory (Map bounded to 500 sensors). Horizontal scaling across multi-node worker clusters requires replacing this with a distributed cache (e.g., Redis).
- **Sandbox Containerization Fallback**: While Docker container isolation (docker run --network none) is architected for Mode B code verification, containerized execution falls back to an isolated child process (runner.js) with restricted environment variables in environments where Docker-in-Docker socket mounting is prohibited.
- **Entity Identification Convention**: Spatial and categorical peer discovery currently relies on a structured naming format (<zone>-<type>). Dynamic graph-based sensor topology lookups via an enterprise asset inventory have not yet been implemented.